COMP1531
Software Engineering Fundamentals

Week 2

Projects - Package Management

In this Lecture

Why?

- To utilise javascript fully, we need to understand how to install **other modules that aren't on our system**
- We need to know how to manage our installations on **multi-user projects**

What?

- Problems with packages
- NPM and how it works
- How to manage packages
- Custom scripts

Problems That We Face

Sometimes we might want to use a library in **NodeJS**, but this library wasn't built-in by us. Someone else on the internet wrote it.

An example of this might be you googling "javascript how do I check if a date is valid" and coming across some code. And we find this snippet that looks good... so we try it out!

```
import { isValid } from 'date-fns';

  function dateIsValid(year, month, day) {
    return isValid(new Date(year, month, day));
  }

  console.log(dateIsValid('2022', '14', '02'));
```

Problems That We Face

But when we run it we get this error...

```
1  SyntaxError: Cannot use import statement outside a module'
```

We need tools to solve this...

Problems That We Face

But when we run it we get this error...

```
1  SyntaxError: Cannot use import statement outside a module'
```

We need tools to solve this...

NPM: Node Package Manager

NPM (Node Package Manager) is a tool that is automatically installed alongside NodeJS to manage dependencies/modules/libraries (all the same thing) for NodeJS (Javascript) projects.

It's command on terminal is:

npm

NPM: Node Package Manager

The most common usage of NPM is to allow you to download external libraries.

The external libraries that you're able to download with NPM are found on the npmjs website

.

Let's have a look for our `date-fns` library!

NPM: Node Package Manager

- To **setup a code repository** to use npm, all we need to do is run `npm init` (if it hasn't already been run). It will ask you a few questions (don't stress about getting them right, you can change them later).
- Once this is done you should now see a `package.json` in your repository. This is essentially your **projects NPM configuration** file.

package_simple.json

```
{
  "name": "example",
  "version": "1.0.0",
  "description": "",
  "type": "module",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}
```

Sometimes we need to configure further. For example for 1531 reasons we added "type": "module" too.

Managing Packages

We can install dependencies with **npm install [dependency]**

For example:

```
npm install date-fns
```

You will see that this command automatically adds the most recent stable version of **`date-fns`** to our `package.json`.

Let's inspect package.json.

Take note of the **~** and **^** symbols.

Managing Packages

Our code will now run successfully.

```
import { isValid } from 'date-fns';

function dateIsValid(year, month, day) {
  return isValid(new Date(year, month, day));
}

console.log(dateIsValid('2022', '14', '02'));
```

date_fns.js

Anatomy of NPM

- The structure of NPM involves a few key files and folders.
    - `package.json`
        - where we store metadata about our project including a list of dependencies to install
    - `package-lock.json`
        - where we store versioning information about dependencies to ensure everyone has the right versions (oversimplification)
    - `node_modules/`
        - Where the dependencies are installed locally.

Anatomy of NPM

The structure of NPM involves a few key files and folders.

- `package.json` - changes are always committed to git.
- `package-lock.json` - changes are always committed to git.
- `node_modules/` - Changes are **never** commited to git.

Anatomy of NPM

- By committing this information we ensure that our environment is completely reproducible on others' machines + our own.
- If you fresh clone the repo, the **`node_modules`** folder won't exist which means the dependencies won't work.
- However, if you run **`npm install`** it will read the `package.json` and `package-lock.json` file to install the appropriate dependencies.

Custom Scripts

A useful feature of npm that we will explore in another lecture is the ability to add scripts to the `package.json`.

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1"
};
```

Custom Scripts

We can add our own command and run it with:

```
npm run bill
```

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "bill": "echo 'Hi Bill!'"
};
```

Scripts use a mixture of *bash* and *json*. We will cover these more later in the course. For now we'll tell you exactly how to modify this stuff.

Feedback

Or go to the form here